Gestiunea Securitatii

- Gestiunea cheilor
  - Stabilirea cheilor
  - Distributia cheilor
- Gestiunea sigura a grupurilor (Secure Group Management)
  - Adaugarea unui nou membru al grupului
- Gestiunea autorizarii
  - delegarea

Gestiunea cheilor

- Generarea, distributia si instalarea cheilor (noile chei independente de cele vechi)

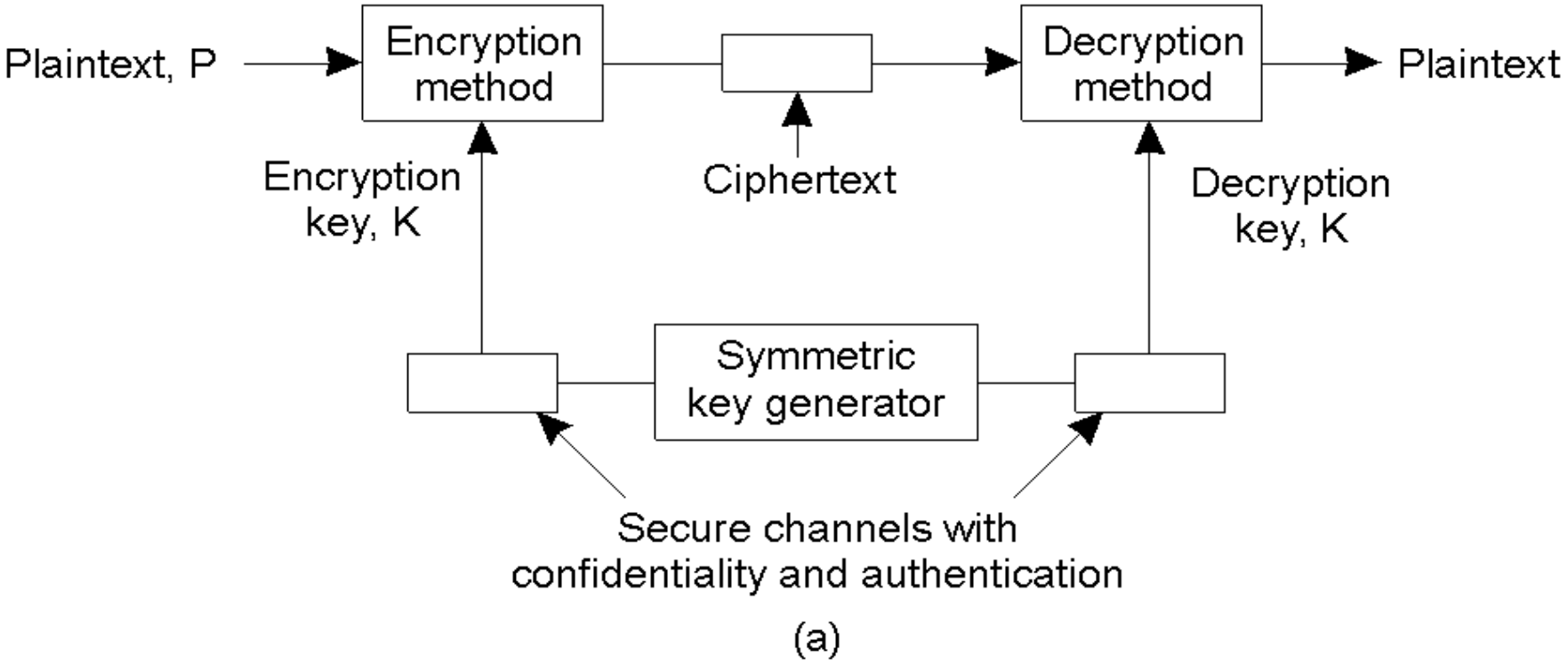
sunt asociate cu

- Identificarea si autentificarea membrilor
- Controlul accesului (operatia de alaturare trebuie validata)

Generarea cheilor

1. Chei secrete:
   - De catre o a treia parte, de incredere (KDC = Key Distribution Center)
   - Fara a treia parte; ex Diffie-Hellman
2. Chei publice:
   - De catre o autoritate de certificare (CA = Certificate Authority)

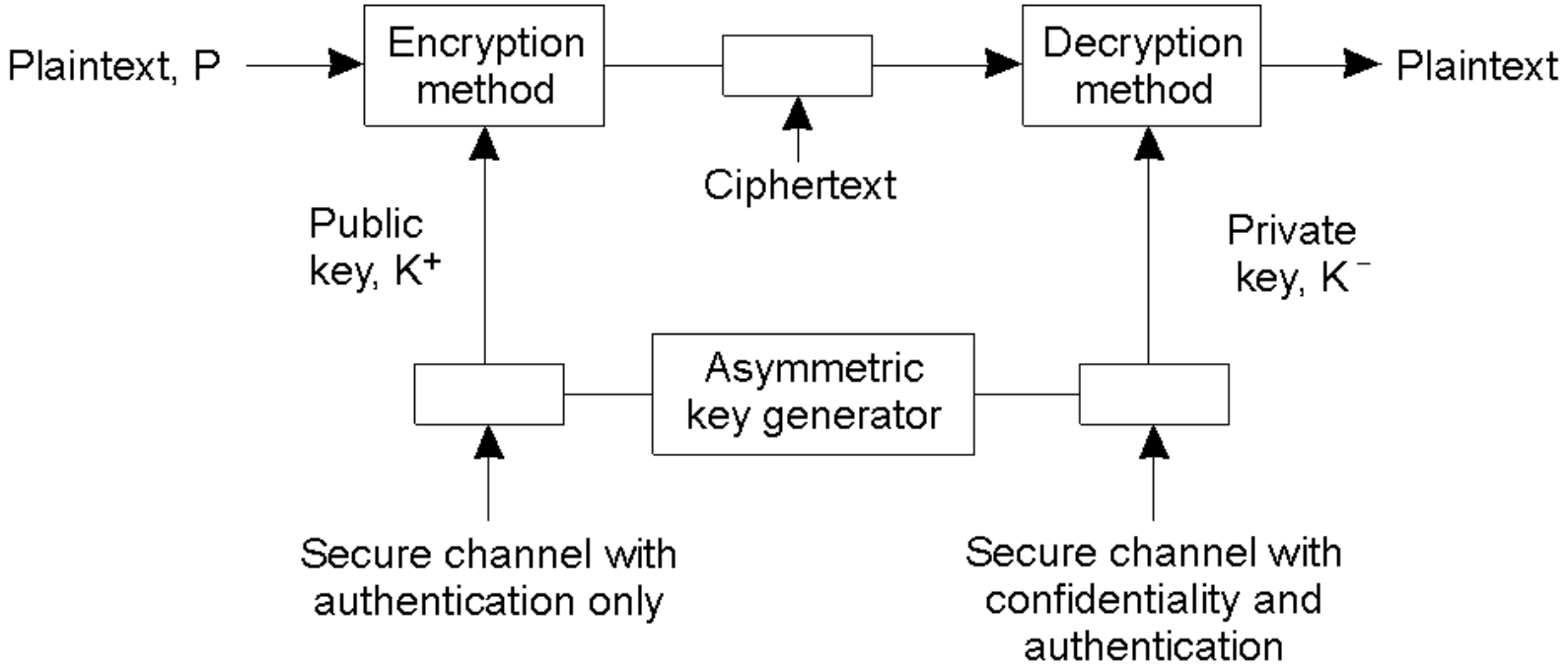
Distributia cheilor secrete



Foloseste cai autentificate si confidentiale

- Telefon, posta, valiza diplomatica, contact direct

Distributia cheilor publice



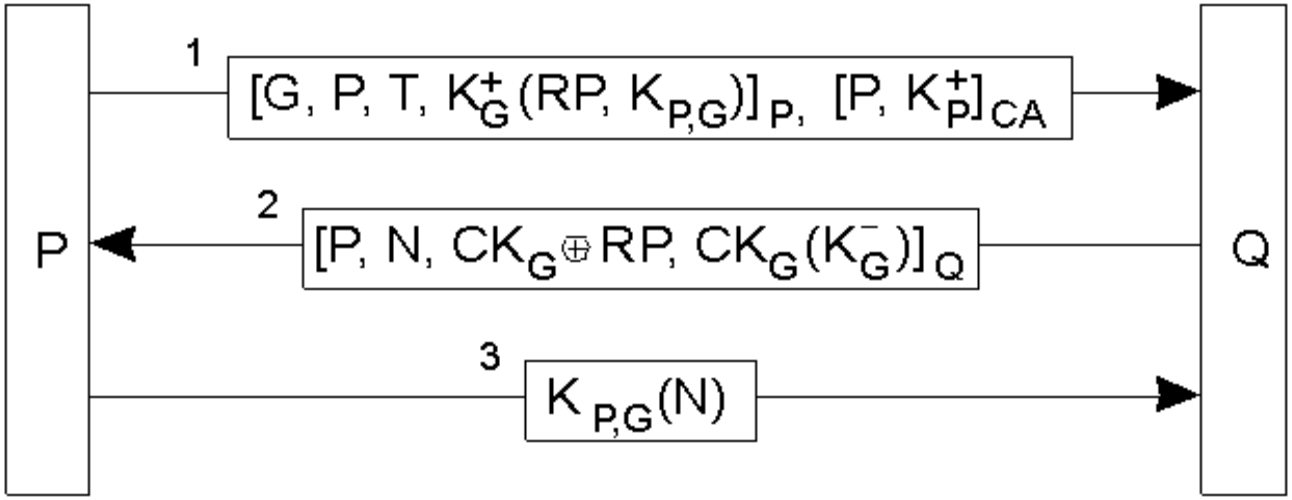
(b)

Cheia publica – prin canale autentificate

- Certificate cu cheie publica

Cheia privata – prin canale confidentiale si autentificate

Gestiunea sigura a grupurilor (1)



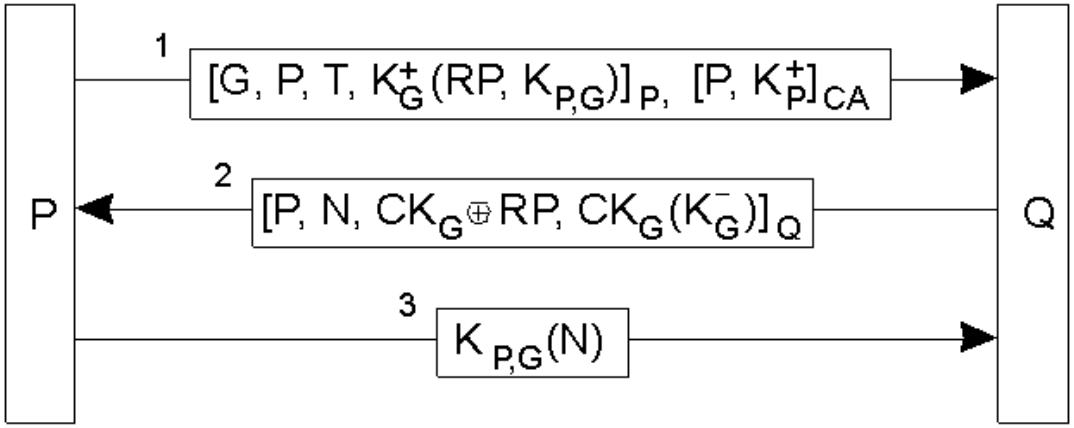
Grupul foloseste K^+_G / K^-_G pentru a comunica cu alte procese din afara

CK_G (secreta) in interiorul grupului

Pentru admiterea sa in grup P trimite mesajul 1:

- o cerere de alaturare la grup, semnata de el
  - T – timpul curent local lui P
  - RP – Reply Pad – folosit la alcatuirea raspunsului
  - $K_{P,G}$ – cheie secreta partajata intre P si G
- si un certificat semnat de CA cu cheia publica K^+_P a lui P

Gestiunea sigura a grupurilor (2)



Q autentifica P si

- Verifica daca P poate fi admis ca membru
- Transmite mesajul semnat 2 de admitere in grup
  - N – Nonce
  - CK_G – cheia secreta a grupului G combinata cu RP
  - K^-_G – cheia privata a grupului criptata cu cheia secreta a grupului CK_G
    - Obs: in mesaj 2, CK_G este criptat cu RP, nu cu K^+_P; motiv - daca K^-_P este compromisa atunci CK_G poate fi compromisa.

P autentifica Q si

- Transmite inapoi N criptat cu cheia secreta partajata $K_{P,G}$

Managementul cheilor pentru comunicarea de grup

Slabiciune metoda prezentata: P poate vedea mesaje anterioare admiterii sale in grup

Uzual, cheia de grup se schimba la modificarea componentei grupurilor:

- Un nou membru nu poate intelege mesajele anterioare intrarii sale in grup
- Un fost membru (care a iesit sau a fost exclus din grup) nu poate intelege mesajele transmise dupa ce a parasit grupul

Managementul cheilor pentru comunicarea de grup

Distribuirea unei noi chei de grup

- Banala la alaturarea unui nou membru
  - Noua cheie se transmite multicast criptata cu cheia veche
- Foarte complicata la iesirea unui membru – schema nescalabila
  - un KDC are o cheie secreta cu fiecare membru din grup
  - Alcatuieste un mesaj lung de O(n): fiecare inregistrare este noua cheie criptata cu cheia secreta partajata cu un membru din grup (face n criptari)
  - Transmite multicast mesajul (mesajul este mare)
  - Fiecare membru isi ia partea criptata cu cheia sa

Protocoale pentru managementul cheilor de grup

Protocoale

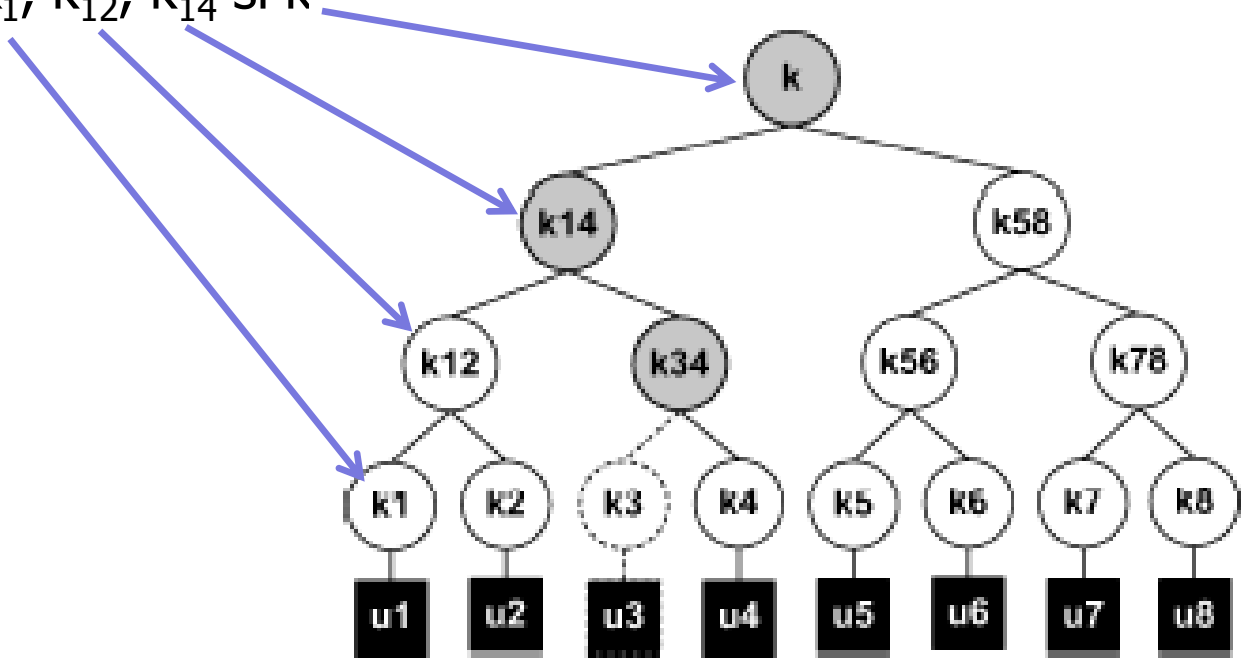
- Centralizate
  - Un singur KDC
- Descentralizate
  - Mai multe KDC pentru subgrupuri
- Distribuite
  - Nu exista un KDC

Management centralizat – LKH (Logical Key Hierarchy)

Propus Wong, Wallner

KDC pastreaza chei organizate logic ca un arbore de chei

- Fiecare nod corespunde unei KEK (Key Encryption Key) cu care se cripteaza alte chei
- Radacina corespunde cheii de grup **k** (cu care se cripteaza datele)
- Fiecare frunza este asociata cu un membru
- Fiecare membru are cheia KEK din frunza si cheile KEK din calea de la el la radacina (in total $\log_2 n + 1$ chei, $\log_2 n$ = adancime arbore balansat)
- Ex: u_1 cunoaste k_1, k_{12}, k_{14} si k



LKH (Logical Key Hierarchy) – nou membru

Membrul nou este asociat cu o frunza si inclus in arbore

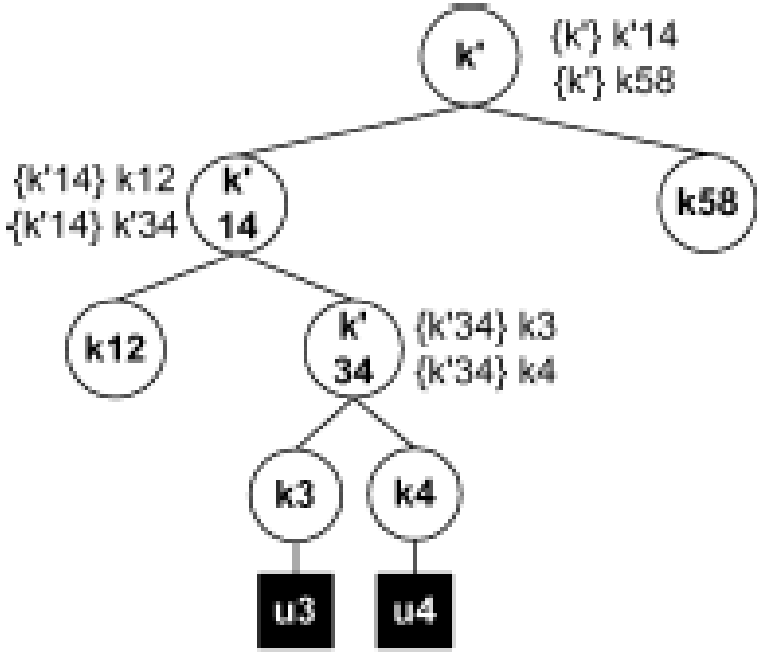
KDC schimba cheia radacinii si toate cheile nodurilor pe calea la noul nod

Se compune mesaj cu noile KEK; un KEK este criptat cu cheile celor doi fii din arbore (lungime mesaj $2\log_2 n$)

Ex: nodul u_3 primeste cheia K_3 si este adaugat la nodul k_{34}.

Cheile k_{34}, k_{14} si k trebuie schimbate

- KDC genereaza k'_{34}, k'_{14} si k'
- Cripteaza fiecare cheie cu cheile fiilor sai
- Compune mesaj de lungime 2*3
- Transmite multicast
- Fiecare membru calculeaza noul set de chei



LKH (Logical Key Hierarchy) – eliminare membru

Ex: eliminare u_4

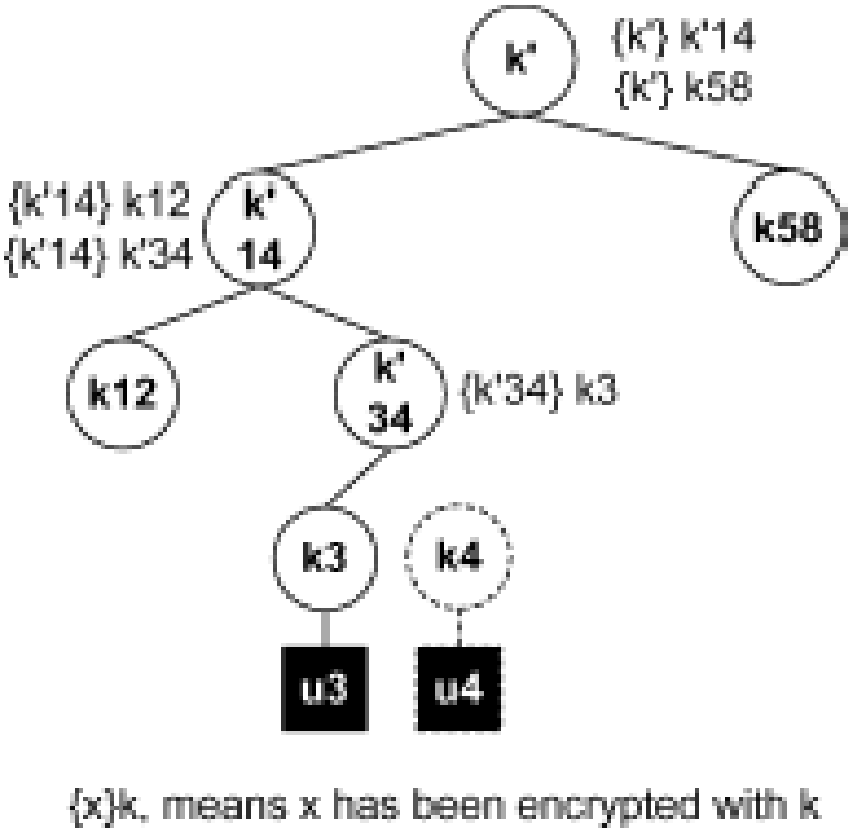
Noile chei sunt criptate cu KEK ale fiilor

Exceptie k'_{34} criptat doar cu cheia fiului ramas, k_3

Imbunatatire Waldvogel – protocol LKH+

Foloseste o functie one-way

Fiecare membru calculeaza noile chei aplicand functia one-way pentru cheile compromise



OFT - One-way Function Tree

Propus McGrew si Sherman

Schema reduce lungimea mesajelor la $\log_2 n$

Cheia unui nod este generata prin "orbirea" cheilor fiilor si mixarea dupa formula:

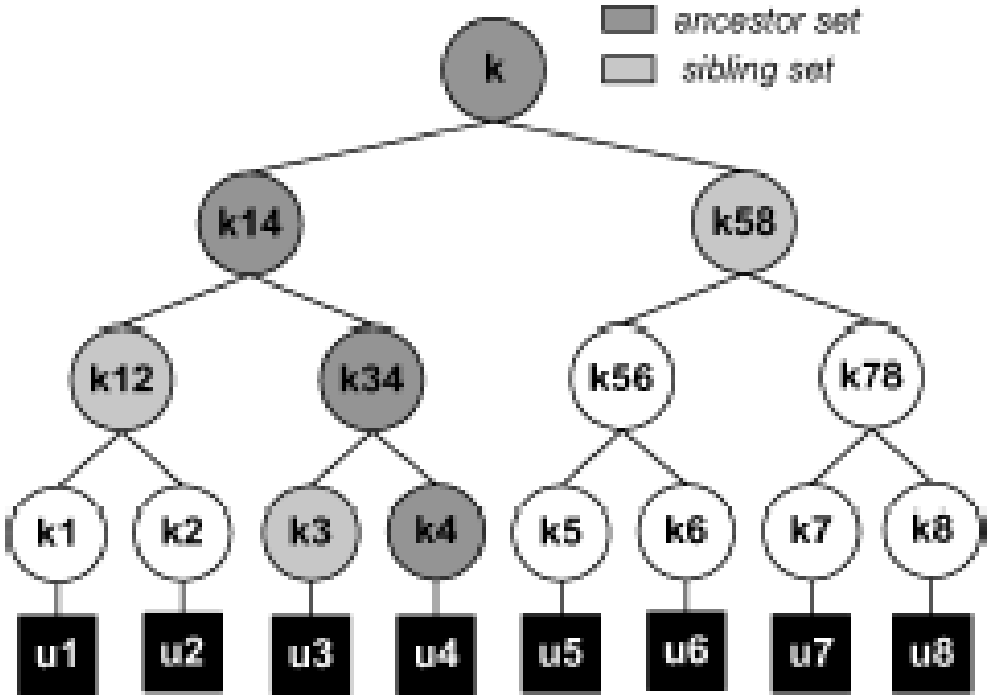
$$K_i = f(g(k_{left(i)}), g(k_{right(i)}))$$

- **left(i), right(i) – fiii lui i**
- **g – functie one-way de "orbire"**
- **f – functie de mixare**

Fiecare nod are asociate doua seturi:

- Stramosi (ancestors) – nodurile de la el la radacina
- Rude (siblings) – fratii nodurilor din setul Stramosi

Ex: seturile pentru u4



OFT (2)

Fiecare membru pastreaza $\log_2 n+1$ chei

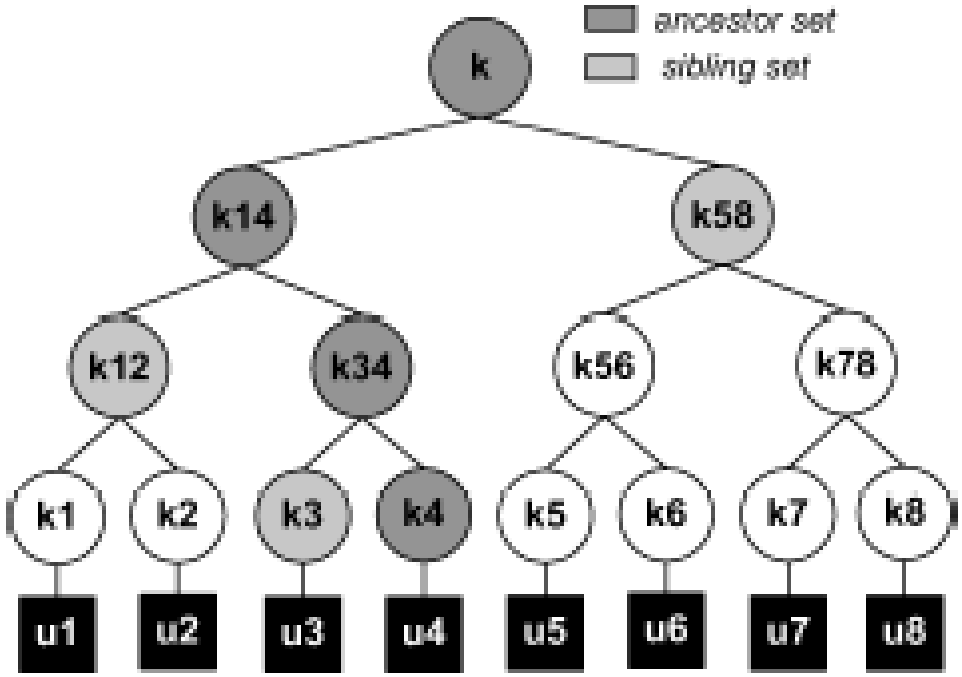
Ex: u_4 pastreaza 4 chei

- Cheia sa (k_4)
- Cheia orbita a fratelui $g(k_3)$
- Cheile orbite ale Rudelor $g(k_{12})$, $g(k_{58})$

Din aceste info, u_4 poate genera cheile ancestorilor:

(cu albastru, info cunoscuta de u_4)

- $k_{34} = f(g(k_3), g(k_4))$
- $k_{14} = f(g(k_{12}), g(k_{34}))$
- $k = f(g(k_{58}), g(k_{14}))$



OFT – nod nou

Ex: u_3 se alatura grupului

Trebuie modificate k_{34}, k_{14} si k

KDC transmite 3 chei orbite $g(k_3)$, $g(k'_{34})$, $g(k'_{14})$ criptate

respectiv cu k_4, k_{12} si k_{58} adica $\{g(k_3)\}k_4$, $\{g(k'_{34})\}k_{12}$, $\{g(k'_{14})\}k_{58}$

Noile chei ale ancestorilor pot fi calculate de fiecare membru al grupului

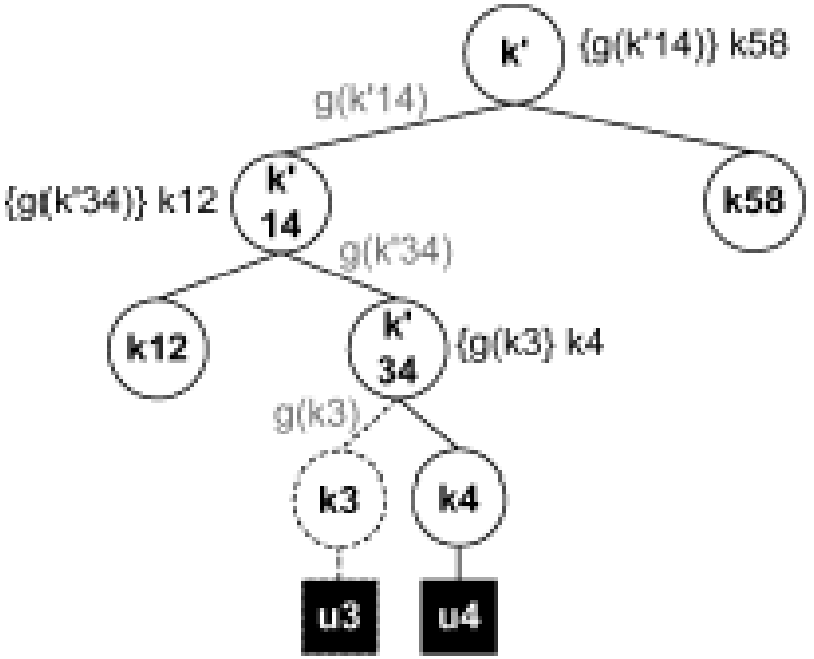
De ex. u_4 poate genera:

$k'_{34} = f(g(k_3), g(k_4))$

$k'_{14} = f(g(k_{12}), g(k'_{34}))$

$k' = f(g(k'_{14}), g(k_{58}))$

[cu rosu valorile stiute dinainte]



{x} k, means x has been encrypted with k

g(x), means x was hash with hash function g

Protocoale descentralizate - Iotus

Propus de Mittra

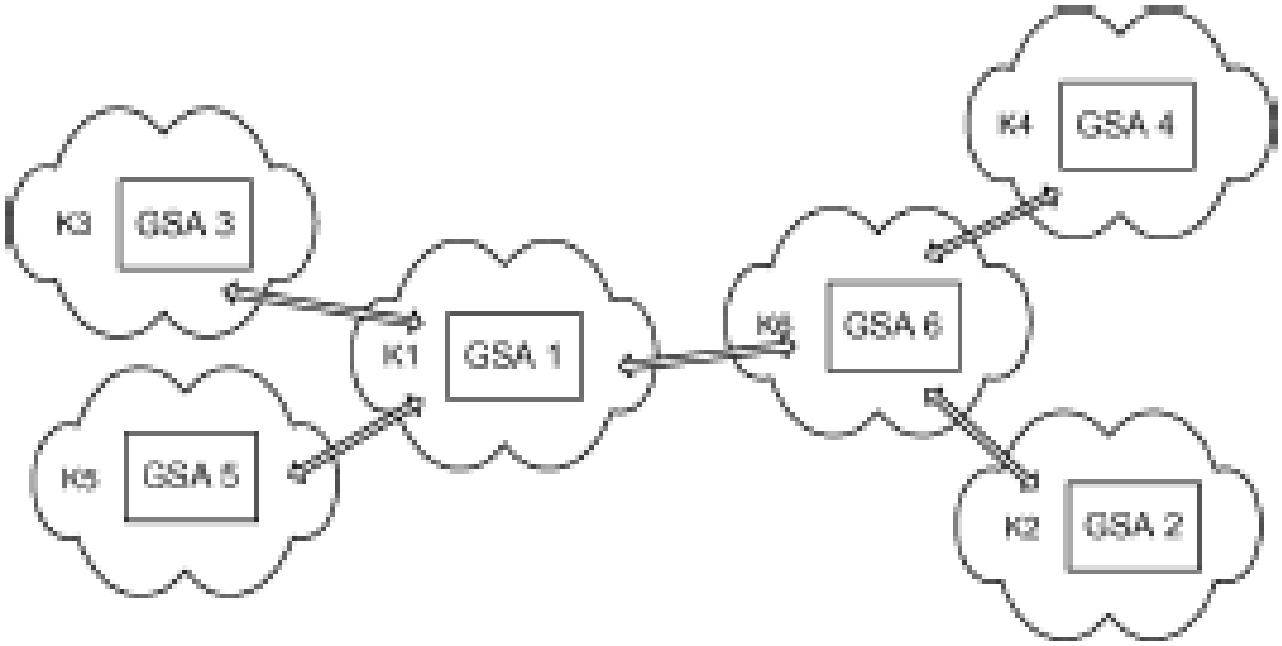
Grupul impartit in sub-grupuri

Fiecare sub-grup gestionat de un GSA – Group Security Agent

GSAs formeaza un grup de nivel inalt gestionat de Group Security Controller

Sub-grupurile au chei diferite, independente

- cheile se schimba la nivelul sub-grupurilor
- Necesita translatarea mesajelor transmise de la un sub-grup la altul



DEP - Dual-Encryption Protocol

Propus de Dondeti

Sub-grupuri, fiecare controlat de un SGM – Sub-Group Manager

- SGM distribuie chei catre membrii sub-grupului sau, dar nu le poate decripta

Trei feluri de KEK

- KEK_{i1} – partajata de **SGM_i** si membrii sub-grupului i
- KEK_{i2} – partajata de **Group Controller - GC** si membrii sub-grupului i fara SGM_i
- KEK_{i3} – partajat de **GC** si **SGM_i**

DEK – Data Encryptioin Key – una singura pentru grup

Distributie DEK

- GC transmite pachete $\{\{DEK\}KEK_{i2}\}KEK_{i3}$
- SGMi recupereaza $\{DEK\}KEK_{i2}$; el nu cunoaste KEK_{i2}
- Cripteaza mesajul $\{\{DEK\}KEK_{i2}\}KEK_{i1}$ si-l trimite sub-grupului i
- Fiecare membru al sub-grupului i decripteaza de doua ori si obtine DEK

Schimbare componenţa sub-grup i

- SGM_i schimba KEK_{i1} si-l transmite membrilor actuali ai sub-grupului
- Urmeaza schimbarea DEK si distributia ei (ca mai sus)
- Membrii exclusi pot primi mesajele grupului pana la schimbarea DEK

MARKS

Propus de Briscoe

Foloseste o cheie de criptare distincta pentru fiecare interval de timp

Cheile sunt frunze intr-un arbore hash generat dintr-o singura sursa

Notatie: $S_{i,j}$ inseamna nodul j la nivelul i din arbore

Calcul chei:

- Calculeaza adancimea D (conform numar total chei N = 2^D)
- Alege aleator radacina $S_{0,0}$
- Genereaza sursele intermediare *left* $S_{1,0}$ si *right* $S_{1,1}$

$S_{1,0} = b(ls(S_{0,0}))$

$S_{1,1} = b(rs(S_{0,0}))$

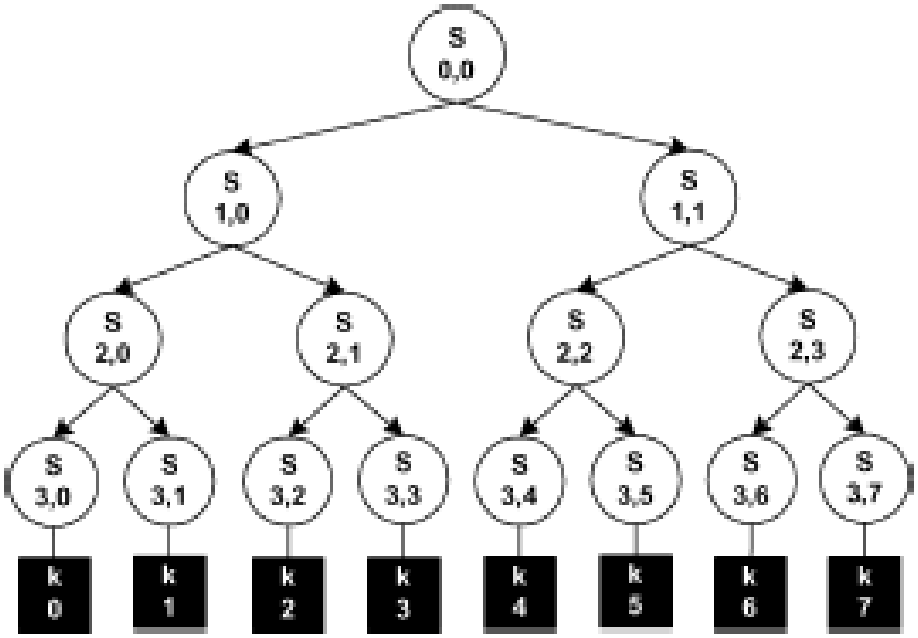
ls = left shift one bit

rs = right shift one bit

b = blinding function

- Genereaza similar celelalte nivele

Un membru care participa la grup in intervalele 3-7 primeste sursele $S_{3,3}$ si $S_{1,1}$



Protocoale distribuite - Group Diffie-Hellman key exchange

Propus de Steiner

Grupul are **n** membri; fiecare membru **i** are un numar secret **xi**

Grupul stabileste doua numere prime (q si a) si incepe sa calculeze distribuit valorile intermediare; protocolul are n runde

- Primul membru calculeaza a^{x1} si o paseaza urmatorului
- Al doilea membru
  - primeste a^{x1} si
  - calculeaza a^{x2} si $a^{x1.x2}$
  - genereaza $\{a^{x1},a^{x2},a^{x1.x2}\}$
  - si transmite celui de al treilea membru
- Fiecare membru primeste un set de valori intermediare si genereaza un nou set folosind numarul sau secret
  - Setul generat de membrul **2** are **2** valori **intermediare** cu **1** exponent si o valoare **cardinal** cu **2** exponenti

Group Diffie-Hellman key exchange

- Setul generat de membrul **i** va avea **i** valori **intermediare** cu **i-1** exponenti si o valoare **cardinal** cu **i** exponenti
- De ex: membrul 4 primeste de la 3 $\{a^{x2.x3}, a^{x1.x3}, a^{x1.x2}, a^{x1.x2.x3}\}$

  si genereaza $\{a^{x2.x3.x4}, a^{x1.x3.x4}, a^{x1.x2.x4}, a^{x1.x2.x3}, a^{x1.x2.x3.x4}\}$

  valoarea cardinal generata este $a^{x1.x2.x3.x4}$
- Membrul n (care are valoarea secreta xn)
  - calculeaza cheia k = $\mathbf{a^{x1.x2.x3...xn} \bmod q}$ din valoarea cardinal $a^{x1.x2.x3...xn-1}$ ridicata la puterea xn
  - Ridica toate valorile intermediare la puterea xn
  - Transmite multicast tot setul (nu include cheia k)
- Fiecare membru **i** extrage valoarea intermediara la care nu s-a folosit **xi** si calculeaza cheia k prin ridicarea la puterea **xi**
  - membrul **2** extrage $\mathbf{a^{x1.x3. \ldots .xn-1.xn}}$ si ridica la puterea **x2** calculand cheia k = $\mathbf{a^{x1.x2.x3...xn} \bmod q}$

D LKH – Distributed Logical Key Hierarchy

Propus de Rodeh

Nu are un Group Controller care sa stie toate cheile

Subarborii stabilesc cheile

Fie subarborii **L** si **R**

Liderii $\mathbf{m_l}$ respectiv $\mathbf{m_r}$

Cheile $\mathbf{K_L}$ respectiv $\mathbf{K_R}$

Protocol

1. m_l alege o noua cheie k_{LR} si o trimite lui m_r pe un canal securizat
2. m_l cripteaza k_{LR} cu k_L si difuzeaza rezultat tuturor membrilor subarbore L

   m_r cripteaza k_{LR} cu k_R si difuzeaza rezultat tuturor membrilor subarbore R

Exemplu:

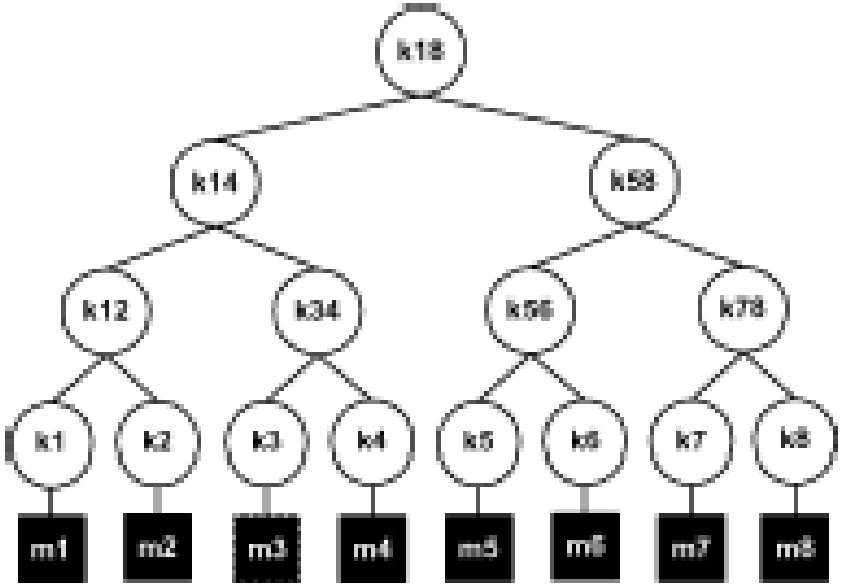
Membrii 1,2 stabilesc cheia **k12**

3,4 stabilesc cheia **k34**

1,2 si 3,4 stabilesc cheia **k14**

….

1,2,3,4 si 5,6,7,8 stabilesc cheia **k18**



DH-LKH - Diffie-Hellman Logical Key Hierarchy

Propus de Perrig si Kim

Foloseste LKH

Diferente

- Membrii genereaza cheile in nivelul superior folosind algoritmul Diffie-Hellman

  $k = a^{kl.kr} \bmod p$

Exemplu

$k_{12} = a^{k1.k2} \bmod p$

$k_{34} = a^{k3.k4} \bmod p$

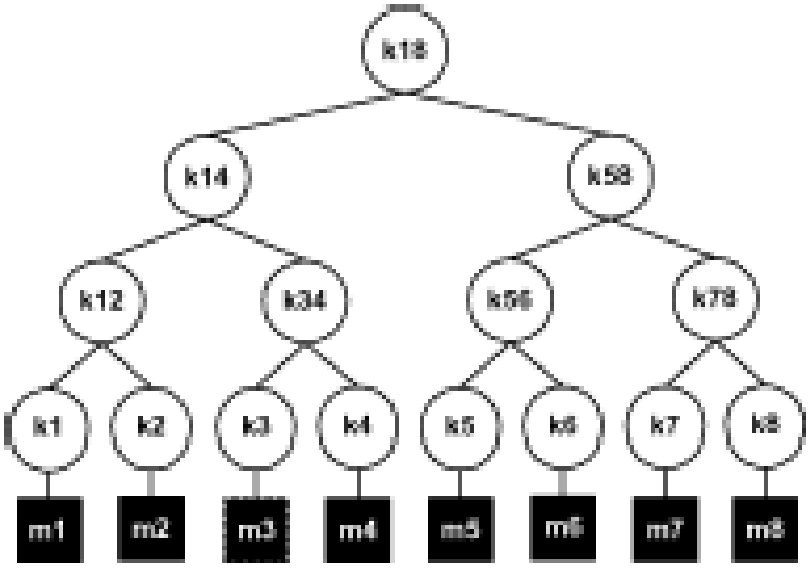
$k_{14} = a^{k12.k34} \bmod p$

...

$k_{58} = a^{k56.k78} \bmod p$

$k_{18} = a^{k14.k58} \bmod p$

Numar runde: $\log_2 n$



Delegare (1)

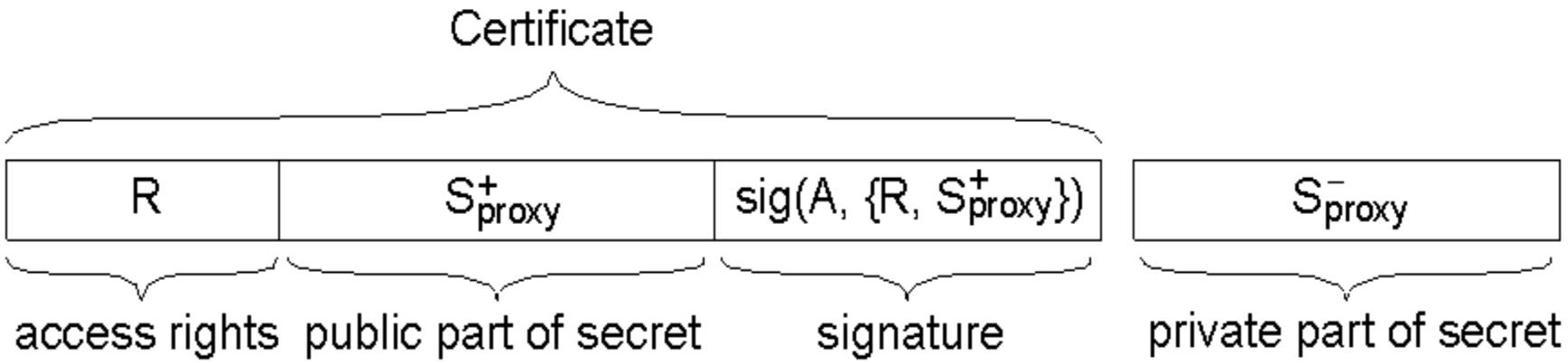
Abordari

- Certificate nominale
  - receptorul trebuie sa primeasca certificatul direct de la garant
  - nu poate fi pasat de receptor unui alt receptor
- Certificate anonime
  - receptorul certificatului il poate folosi pentru delegarea drepturilor catre un alt receptor
  - necesita protectie contra copierii ilegale

Proxy

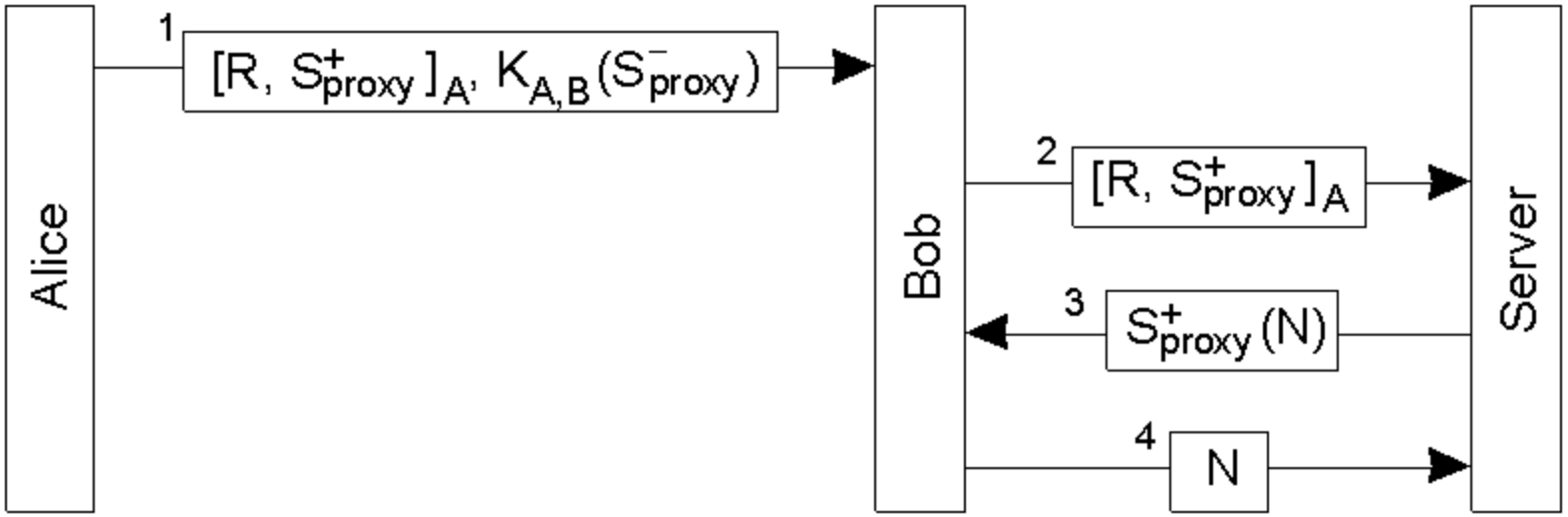
Proxy – token (semn) care permite posesorului sa opereze cu aceleasi drepturi (sau drepturi mai restranse) ca garantul

- token include un **certificat** si un **secret** generate de **garantul** A
  - R – drepturi de acces
  - S^{+}_{proxy} – partea publica a secretului folosit pentru a autentifica detinatorul certificatului (“intrebarea”)
  - S^{-}_{proxy} – partea privata (“raspunsul” secret)



Distributed Systems, Tanenbaum

Utilizare proxy pentru a delega si proba proprietatea asupra drepturilor de acces



Semnaturi duale

Dual signature (semnat de Alice si transmis lui Bob)

Separa **ordinul** de informatia de **plata**

– recipientul ordinului nu are acces la informatia de plata

m1 denota ordinul si

m2 este informatia de plata

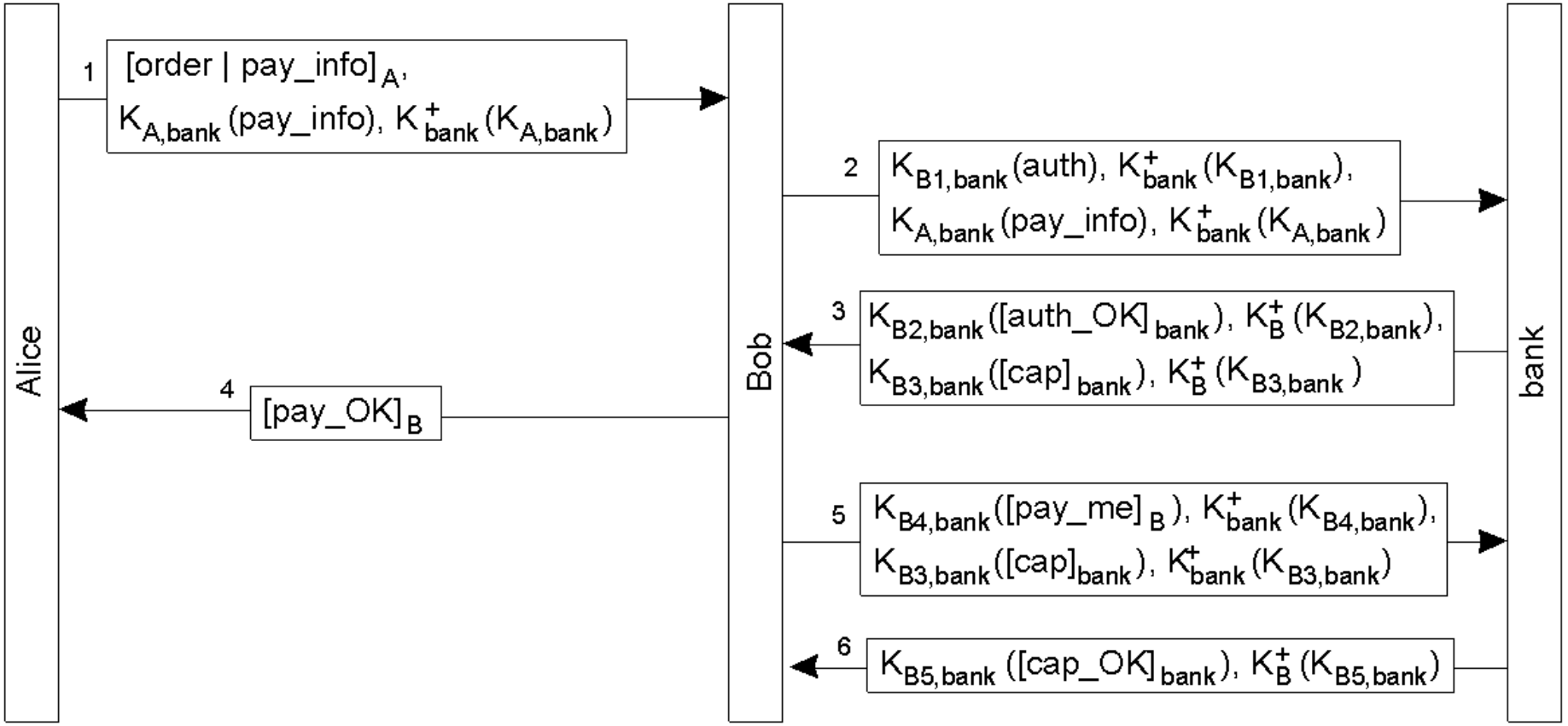
Semnatura duala a mesajelor m1 si m2

$$[m1 \mid m2]_A = [m1, H(m2), H(concat(m1, m2)), K^{-A}(H(concat(m1, m2)))]$$

Proprietati

- doar m1 (ordinul) este dezvaluit lui Bob
- Bob poate autentifica pe Alice (folosind semnatura)
- Bob poate testa ca Alice a semnat intregul mesaj

Criptografia in Secure Electronic Transaction



$K_{A,bank}$, $K_{Bi,bank}$ – chei de sesiune

K^-_{bank} / K^+_{bank} – chei privata / publica

auth – cerere de autorizare

auth_OK – mesaj autorizare

cap – mesaj captura

pay_OK – confirmare